**Atividade Prática: Expansão do Gerenciador de Despesas**

**Contexto da Atividade**

Na aula, foi desenvolvida a estrutura base de um aplicativo de controle financeiro com a listagem de despesas (TodasDespesas.js, DespesasRecentes.js), calculando o total gasto via .reduce() e permitindo o cadastro/edição de despesas (GerenciarDespesa.js).

O objetivo desta atividade é **expandir o projeto** incorporando o conceito de **Categorização de Despesas** e um **Filtro Dinâmico de Exibição**, mantendo a arquitetura de componentes reutilizáveis criada em sala.

**Requisitos Práticos**

**Tarefa 1: Atualização do Modelo de Dados e Formulário (GerenciarDespesa.js)**

1. **Inclusão do campo Categoria:**
    - Adicione o atributo categoria aos objetos do array de despesas (ex: 'Alimentação', 'Transporte', 'Lazer', 'Contas').
    - No arquivo GerenciarDespesa.js, crie um novo estado com useState para gerenciar a categoria selecionada (const [categoria, setCategoria] = useState('')).
    - Adicione um componente de seleção de categoria na tela (pode ser implementado utilizando componentes Pressable dispostos lado a lado ou um menu de seleção).
2. **Validação de Formulário:**
    - Mantenha a função de tratamento de valor com Regex (/^\d*\.?\d{0,2}$/) para limitar a duas casas decimais.
    - Mantenha o fluxo de seleção de data utilizando o @react-native-community/datetimepicker.
    - Valide se os campos (descrição, valor e categoria) foram preenchidos antes de permitir a submissão dos dados.

**Tarefa 2: Customização dos Componentes visuais (DespesaItem.js e DespesaSumario.js)**

1. **Aprimoramento do Item (DespesaItem.js):**
    - Altere o retorno do DespesaItem.js para exibir a **categoria** junto com a data (formatada via getDataFormatada), a descrição e o valor.
    - Crie uma estilização no StyleSheet para destacar visualmente a categoria (ex: uma tag com fundo cinza claro e bordas arredondadas).
2. **Cálculo de Totais (DespesaSumario.js):**
    - Garanta que a constante somaDespesas continue utilizando o método .reduce() para somar os valores das despesas recebidas via props:

$$\text{total} = \text{despesas.reduce}((\text{acumulador}, \text{item}) \Rightarrow \text{acumulador} + \text{item.valor}, 0)$$

    - Mantenha a formatação monetária utilizando .toFixed(2) antecedido por R$.

**Tarefa 3: Filtro Dinâmico por Categoria**

1. **Filtragem de Arrays:**
    - Crie uma função de filtragem que utilize o método .filter() para retornar apenas as despesas pertencentes à categoria selecionada pelo usuário.
2. **Reutilização do DespesaSaida.js:**
    - Passe a lista filtrada como prop para o componente DespesaSaida.js.
    - A FlatList interna de DespesaLista.js deve re-renderizar automaticamente os itens e o DespesaSumario.js deve recalcular a soma apenas da categoria filtrada.

**Estrutura do Projeto Esperada**

Os arquivos desenvolvidos devem manter a organização modular apresentada na aula:

```
Plaintext

src/
├── components/
│   └── despesa/
│       ├── DespesaItem.js
│       ├── DespesaLista.js
│       ├── DespesaSaida.js
│       └── DespesaSumario.js
└── screens/
    ├── TodasDespesas.js
    ├── DespesasRecentes.js
    └── GerenciarDespesa.js
```

**Critérios de Avaliação**

| Critério | Peso | Detalhes |
|---|---|---|
| **Componentização & Props** | 25% | Passagem e desestruturação correta de propriedades entre componentes (despesas, periodo). |
| **Manipulação de Arrays** | 25% | Uso adequado de .reduce() para calcular somatórios e .filter() para aplicar o filtro por categoria. |
| **Gerenciamento de Estado & Formulário** | 25% | Funcionamento de useState, TextInput (keyboardType='decimal-pad'), DateTimePicker e validações. |
| **Exibição & Interface** | 15% | Uso do FlatList com keyExtractor e customização de estilos via StyleSheet. |
| **Organização do Código** | 10% | Separação correta de arquivos e legibilidade do |

| | | |
|---|---|---|
| | | código. |

**Desafio Extra (Bônus)**

Adicione um indicador visual no DespesaSumario.js: se o somatório das despesas exibidas ultrapassar **R$ 200,00**, altere a cor do texto do valor para vermelho; caso contrário, mantenha a cor padrão.